

LEVEL 2 · INTERMEDIATE

Prompt Engineering: From Composing to Directing

You can already build a strong prompt. Now learn to steer it — with examples, reasoning, structure, and your own documents.



YOUR LEARNING JOURNEY

From asking to designing — what each level of the program builds

YOU ARE HERE

1

STAGE 1

Asking

One-line requests, hit or miss. The everyday habit this program upgrades.

2

STAGE 2 · DONE

Composing

CO-STAR to build strong prompts on purpose — then the Refine loop to improve them in one follow-up.

3

STAGE 3 · INTERMEDIATE

Directing

Show it examples, ask it to reason first, structure long prompts, and ground answers in your own BJ's documents.

4

STAGE 4 · ADVANCED

Designing

Chain tasks into steps, build reusable team templates, engineer context, and evaluate outputs with guardrails.



RECAP · WHERE YOU LEFT OFF

You can already compose a strong prompt

In Beginner you stopped typing one-liners and hoping. You learned to build prompts on purpose with CO-STAR — Context, Objective, Style, Tone, Audience, Response — and the Refine loop: check the answer, then improve it in one follow-up instead of starting over.

Today we go one step further. Same six-part habit, but now you actively direct the model: show it what you want, ask it to reason, and feed it your own material.

YOUR STARTING POINT

CO-STAR the six parts of a strong prompt.

🔄 **Refine** improve in one follow-up — don't restart.



LEARNING OBJECTIVES

By the end of this module, you will be able to:

1

Show it examples

Give 1–2 worked examples to lock in the exact style and format.

2

Ask it to reason first

On judgment calls, have it think through the factors before answering.

3

Structure a long prompt

Separate your instructions from your material so they don't blur.

4

Work from your own documents

Ground answers in a real source, quote it — and keep member data safe.

5

Diagnose and refine

Work out why an output missed and fix the prompt, not just re-roll.



WHY THIS MATTERS

From a good answer to a reliable one

Composing gets you a strong answer once. Directing gets you a good answer every time — on the messy, higher-stakes work you actually do: making sense of a pile of member feedback, deciding what to escalate, summarizing a document without inventing facts.

These are small moves. Each takes seconds, and each changes the result completely.

OUR RUNNING EXAMPLE

Last weekend's post-visit survey — about 30 comments.

Five of them:

“Waited 22 min at checkout — only 3 lanes open.”

“Rotisserie chicken is always fresh!”

“The app froze when I clipped the gas coupon.”

“Gas pump 4 is down again.”

“Wilted spinach and moldy berries in produce.”

We'll turn these real comments into something a manager can act on.



SHOW, DON'T JUST TELL

Examples teach faster than instructions

Describing the format you want is slow and easy to misread. Showing it is instant. Give the model one or two entries done exactly right and it copies the pattern for the rest — the way you'd train a new associate by showing them a couple, not reading them the manual.

One or two good examples is usually enough. It's one of the most reliable moves in prompting.

IN PRACTICE

Here are two comments formatted the way I want:

Checkout waits — Negative — 'only 3 lanes open.'

Rotisserie chicken — Positive — 'always fresh.'

Now format the remaining six the same way.



SHOW, DON'T JUST TELL

Told vs. shown — the actual output

TOLD IT — “format these neatly”

THE PROMPT

Format these comments neatly.

WHAT YOU GET BACK

- checkout was really slow saturday, 3 lanes open

The rotisserie chicken is great, always fresh!

- app froze when I clipped the gas coupon

Gas pump 4 down again, ugh.

wilted spinach and moldy berries in produce

WHY IT FALLS SHORT

'Neatly' is a wish, not a format — with nothing to copy, it picked a different shape for every row.

SHOWED IT — two examples first

THE PROMPT

Here are two done right:

Checkout waits — Negative — 'slow.'

Rotisserie chicken — Positive — 'fresh.'

Now do the rest the same way.

WHAT YOU GET BACK

Checkout waits — Negative — 'only 3 lanes open.'

Rotisserie chicken — Positive — 'always fresh.'

App issues — Negative — 'froze on the gas coupon.'

Gas pump — Negative — 'pump 4 down again.'

Produce — Negative — 'wilted spinach, moldy berries.'

WHY IT WORKS

Two finished rows define the format better than a paragraph of rules — it simply continued the pattern it could see.



LET IT THINK FIRST

Ask for the reasoning before the answer

On judgment calls — what to prioritize, what to escalate — a model that answers instantly often grabs the loudest item and misses the important one. Ask it to think through the factors first, then decide. The quality of the final call jumps.

As simple as adding one line about what to weigh before it ranks.

IN PRACTICE

From this feedback, which three issues should we escalate today?

First reason through each one — safety, how many members it affects, and how easy it is to fix — then give me a ranked shortlist.



LET IT THINK FIRST

Answer first vs. reason first

ANSWER FIRST

THE PROMPT

Which 3 issues should we escalate today?

WHAT YOU GET BACK

1. Checkout waits (most comments)
2. Produce quality
3. App issues

(Picks the most-repeated. The gas pump — a safety issue mentioned once — doesn't make the list.)

WHY IT FALLS SHORT

Answering instantly, it grabbed the loudest signal — repetition — because nothing told it what else to weigh.

REASON FIRST

THE PROMPT

...First reason through safety, how many members, and effort — then rank.

WHAT YOU GET BACK

Reasoning: pump 4 is a safety risk; checkout is the most-mentioned (12 of ~30); app issues are low-risk.

Rank: 1) Gas pump (safety) 2) Checkout waits (12) 3) App issues.

WHY IT WORKS

Naming the factors forced them into the decision: safety got weighed before volume, and the ranking shows its work.



STRUCTURE A LONG PROMPT

Keep your instructions and your material apart

When you paste a wall of text under your request, the model can't always tell where your instruction ends and the material begins — so it summarizes the wrong thing or misses half. Put a clear header on each part. A simple label for your task and another for the content is enough to keep them separate.

The longer and messier the input, the more this matters.

IN PRACTICE

YOUR TASK

Group the comments below into themes and count each.

MEMBER COMMENTS

(the full list pasted here)



STRUCTURE A LONG PROMPT

One big blob vs. clearly labelled

ONE BIG BLOB

THE PROMPT

Group these into themes and count them. waited 22 min at checkout... chicken always fresh... pump 4 down... (+25 more)

WHAT YOU GET BACK

Sure — you'd like me to group and count these. The first few mention checkout waits and the rotisserie chicken, which sound positive overall...

WHY IT FALLS SHORT

With no boundary, your task and the comments blur into one wall of text — it can't tell orders from material.

CLEARLY LABELLED

THE PROMPT

*### TASK — group into themes, count each.
COMMENTS — (the 30 pasted below)*

WHAT YOU GET BACK

Checkout waits — 12
Produce quality — 7
Fresh food — 5
App issues — 4
Gas pump — 1

WHY IT WORKS

The labels draw the line: everything under TASK is instruction, everything under COMMENTS is data. Nothing leaks.



WORK FROM YOUR OWN DOCUMENTS

Ground it in the source — and quote it

Ask a model from memory and it can answer confidently but wrong. Give it the actual document, tell it to answer only from that, quote the exact line, and say when something isn't there. Made-up facts drop sharply.

A safety note: only paste material you're allowed to — never a member's personal details or confidential data — and use the tools BJ's has approved.

IN PRACTICE

Using only the vendor one-pager below, list the delivery terms. Quote the exact line for each. If a detail isn't stated, say 'not in the document' — don't guess.



WORK FROM YOUR OWN DOCUMENTS

From memory vs. grounded in the doc

FROM MEMORY

THE PROMPT

What are this vendor's delivery terms?

WHAT YOU GET BACK

Typically 3–5 business days, with free returns within 30 days.

(None of this appears in the document.)

WHY IT FALLS SHORT

Asked from memory, it fills the gaps with plausible patterns — fluent, confident, and never checked against anything.

GROUNDED IN THE DOC

THE PROMPT

Using only the one-pager below, list the delivery terms. Quote each. Say 'not stated' if missing.

WHAT YOU GET BACK

- 'Orders ship within 10 business days.'
- 'Freight billed at cost.'
- Return window — not stated in the document.

WHY IT WORKS

'Only the document, quote it, or say not stated' leaves nowhere to invent — every line can be checked in seconds.



DIAGNOSE, DON'T RE-ROLL

When it misses, fix the prompt on purpose

The beginner reflex is to run it again and hope. The better habit is to name why it missed — too generic? no numbers? wrong format? — and change exactly that part of the prompt. One targeted fix beats five random retries.

This is the Refine loop from Beginner, done deliberately.

IN PRACTICE

Diagnosis: the summary was too vague — no themes, no counts.

Fix: 'Group by theme, tell me how many members mentioned each, and keep it to the top five.'



DIAGNOSE, DON'T RE-ROLL

Flat summary vs. diagnosed and fixed

FLAT SUMMARY

THE PROMPT

Summarize the feedback.

WHAT YOU GET BACK

Members had mixed feedback about their visit — some positive comments and some complaints. Overall it was a fairly typical weekend.

WHY IT FALLS SHORT

A vague ask re-rolled stays vague — the prompt, not luck, was the problem, so retrying changes nothing.

DIAGNOSED & FIXED

THE PROMPT

Too vague. Group by theme, count each, and give the top five.

WHAT YOU GET BACK

1. Checkout waits — 12
2. Produce quality — 7
3. Fresh food — 5 (+)
4. App issues — 4
5. Gas pump — 1 (safety)

WHY IT WORKS

You named the exact gap — themes, counts, a cap — and changed only that. One targeted fix beat five retries.



LIVE DEMO · BEFORE

The undirected ask

▶ PASTE THIS INTO CLAUDE

Summarize this member feedback.

[the weekend's member comments pasted here]

WHAT YOU GET BACK

Members shared mixed feedback this weekend. Some praised the fresh food, while others were frustrated by long checkout lines and problems with the app and the gas station. Overall, sentiment was mixed.

→ **True — but nothing to act on. No themes, no counts, no priority.**



LIVE DEMO · AFTER

The same batch, directed

▶ PASTE THIS INTO CLAUDE

TASK

Group the comments into themes. For each: theme, how many members, sentiment, one quote. Two done right:

Checkout waits — 12 — Negative — 'only 3 lanes open.'

Rotisserie chicken — 5 — Positive — 'always fresh.'

Then reason through which single issue to escalate today (safety, volume, effort) and name it.

COMMENTS

[the batch]

WHAT YOU GET BACK

THEMES

- Checkout waits — 12 — Neg — 'only 3 lanes open.'
- Produce quality — 7 — Neg — 'wilted spinach.'
- Fresh food — 5 — Pos — 'chicken always fresh.'
- App issues — 4 — Neg — 'froze on the coupon.'
- Gas pump — 1 — Safety — 'pump 4 down again.'

ESCALATE TODAY: Gas pump (safety) — rare but a risk. Then checkout staffing (highest volume).

→ Themes, counts, and a priority you can defend — ready for a manager.



WHAT YOU CAN NOW DO

You can direct the model, not just ask it



Show it examples

1–2 done right locks the style and format.



Ask it to reason first

Better calls on what to prioritize and escalate.



Structure a long prompt

Your instructions and material stay separate.



Work from your documents

Grounded, quoted — and member data stays safe.



Diagnose and refine

Name why it missed; fix that one thing.



Next stop: Designing

Advanced — chaining, templates, evaluation.



KEEP-AT-YOUR-DESK CARD

Your Intermediate moves, one card

1

Examples Show 1–2 done right — it copies the pattern.

2

Reason first On judgment calls: “think it through, then answer.”

3

Structure Separate your task from your material with clear headers.

4

Your documents Answer only from the source, quote it, flag what's missing — keep member data out.

5

Diagnose Name why it missed; fix that, don't re-roll.

▶

Directing You're now telling the model what only you know.



TRY THIS ON MONDAY

Point it at one real pile of text

Take a batch of feedback, emails, or notes from your own club. Show it two examples of the format you want, ask it to reason before it ranks, and keep your instructions separate from the material.

OPTIONAL — GO DEEPER

Appendix

Optional extras — a reusable-template habit and where it leads. Dip in when you want to go deeper.



OPTIONAL

Turn a good prompt into a reusable template

Once a prompt works, don't retype it. Save it with blanks for the parts that change — the club, the document, the date — and reuse it. One good prompt becomes a tool your whole team can share.

This is the bridge into Advanced, where reusable templates are the main event.

THE IDEA

Group the **[FEEDBACK SOURCE]** from **[CLUB / DATE]** into themes, count each, and give me the top five with one example quote per theme.



EXAMPLE

Retype it every time vs. fill in the blanks

EVERY TIME FROM SCRATCH

THE PROMPT

(a new, hand-written request each week)

WHAT YOU GET BACK

Week 1: 'Summarize the checkout feedback...'

Week 2: different wording, forgets the counts.

Week 3: another order again.

WHY IT FALLS SHORT

Each retype is from memory, so parts drop off and the wording drifts — and the results drift with it.

ONE SAVED TEMPLATE

THE PROMPT

Group [SOURCE] from [CLUB / DATE] into themes; count; top five.

WHAT YOU GET BACK

Swap [CLUB]=124, [DATE]=Jul 6 →

same five-theme, counted summary — every week, and a teammate can run it too.

WHY IT WORKS

The saved wording never drifts; only the blanks change. Same quality every week, from anyone on the team.